Data Cleaning, Transformation and Modelling Assignment

Theoretical Questions

Question 1: What is Data Loading in Power BI and why is it considered the first step of analysis?

Answer:

Definition of Data Loading

Data Loading in Power BI is the process of connecting to one or more data sources, extracting the required data, applying necessary transformations and cleaning operations in Power Query, and bringing the final prepared data into the Power BI data model where it becomes available for building relationships, writing DAX measures, and creating visualizations. It is the foundational process that makes all subsequent analytical work possible.

Without data loading, Power BI has no information to work with. Every chart, every KPI card, every dashboard, and every DAX calculation depends entirely on data that has been successfully loaded into the model.

Why Data Loading is Considered the First Step of Analysis

Reason 1: No Analysis Can Begin Without Data Power BI is a visualization and analytics tool, not a data creation tool. It cannot generate insights from nothing. Before any report page can be designed, before any measure can be written, and before any relationship can be defined, data must first be extracted from a source and loaded into the model. Data loading is therefore not just the first step but the prerequisite for every other step in the entire Power BI workflow.

Reason 2: The Quality of Loaded Data Determines the Quality of All Analysis Data loading is not merely a mechanical transfer of rows and columns. During the loading process, Power Query is used to clean the data, fix data types, remove nulls, eliminate duplicates, standardize formats, and apply transformation logic. If these steps are skipped or done poorly, every visualization, every aggregation, and every business insight derived from that data will be built on a flawed foundation. This is why data loading is treated as a critical analytical step and not just a technical one.

Reason 3: It Defines the Structure of the Entire Data Model When data is loaded, Power BI assigns data types to each column, creates the table structure, and makes fields available for relationship building in Model View. The decisions made during loading, such as which columns to include, how to name them, what data types to assign, and how to handle missing values, shape the entire analytical model. Changing these structural decisions after the model is built and reports are created is significantly more complex and disruptive. Getting loading right the first time ensures the model is solid before any analysis is layered on top.

Reason 4: It Establishes the Data Refresh Foundation In Power BI, scheduled data refresh in the Power BI Service is built entirely on the connections and transformations defined during the initial data loading process. The loading step records every transformation applied in Power Query as a replayable sequence of steps. Every time the data refreshes, Power BI replays these exact steps against the source. If the loading step is not properly configured, scheduled refresh will either fail or deliver incorrect updated data to live dashboards.

Reason 5: It Enables All Downstream Steps Every capability of Power BI depends on data being loaded first. Relationships between tables can only be built after tables are loaded. DAX measures can only reference fields that exist in loaded tables. Visualizations can only display values that are present in the model. Filters and slicers can only operate on loaded columns. Data loading is the single enabler that unlocks every other feature of Power BI, which is precisely why it is recognized as the first and most foundational step of the entire analysis process.

Question 2: Explain the difference between “Load” and “Transform Data” in Power BI.

Answer:

When connecting to a data source in Power BI Desktop, the Navigator window presents two options at the bottom before the data enters the model. These two options are **Load** and **Transform Data**. While both ultimately bring data into Power BI, they serve fundamentally different purposes and lead to very different outcomes depending on the condition of the data being imported.

Load

Load is the direct import option that takes the data exactly as it appears in the source and brings it straight into the Power BI data model without any intermediate cleaning or transformation step. When the analyst clicks Load, Power BI reads the selected tables or sheets from the source, assigns default data types to each column based on its best guess of the values present, and makes the data immediately available in Data View, Model View, and Report View.

This option is appropriate only when the data is already clean, correctly typed, and fully prepared before it reaches Power BI. It is the faster of the two options since it skips the Power Query Editor entirely and moves directly to making the data available for reporting.

However, choosing Load without verifying the data carries significant risk. If the source data contains nulls, inconsistent formats, incorrect data types, or duplicate records, all of these issues will be loaded directly into the model as they are. There is no opportunity to intercept and correct problems, meaning every report and dashboard built on that data will inherit those flaws silently.

When to use Load: Use Load only when the data source is already clean and validated, such as when connecting to a well-maintained SQL database view that has been pre-processed, or when the data has already been cleaned externally before being connected to Power BI.

Transform Data

Transform Data is the option that opens the **Power Query Editor** before the data is loaded into the model. Instead of importing the data immediately, Power BI presents it in an intermediate editing environment where the analyst can inspect every column, apply cleaning operations, fix data types, remove duplicates, handle nulls, rename columns, split or merge fields, filter rows, and apply any other necessary transformation before the data is committed to the model.

Every transformation step applied in Power Query is recorded as a sequential query step in the Applied Steps panel on the right side of the editor. These steps are saved as part of the Power BI file and are automatically replayed every time the data is refreshed, meaning the same cleaning logic is applied consistently on every future data load without manual intervention.

Transform Data is the recommended approach for the vast majority of real-world datasets because raw data from files, databases, or online services almost always requires some degree of cleaning or restructuring before it is fit for reporting.

When to use Transform Data: Use Transform Data whenever the source data needs any form of cleaning, type correction, null handling, column renaming, format standardization, or structural reshaping before it enters the model. This is the default best practice for most data loading scenarios.

Side-by-Side Comparison

Factor	Load	Transform Data
Opens Power Query Editor	No	Yes
Data enters model immediately	Yes	Only after clicking Close and Apply
Cleaning and transformation possible	No	Yes
Data type correction	Automatic best guess only	Manual control over every column
Null handling	Not performed	Can be applied before loading
Duplicate removal	Not performed	Can be applied before loading
Transformation steps recorded	No	Yes, replayed on every refresh

Factor	Load	Transform Data
Recommended for raw data	No	Yes
Recommended for pre-cleaned data	Yes	Optional
Risk of loading bad data	High	Low

Practical Example Using the Real Estate Dataset

When loading the Real Estate dataset containing Property_ID, Property_Name, City, Price, Area_sqft, Bedrooms, Bathrooms, Owner_Name, and Contact, the difference between the two options plays out as follows.

If **Load** is chosen directly, Power BI assigns data types automatically. It may assign the Contact column a numeric type since it contains phone numbers that look like integers, which would be incorrect since phone numbers should never be used in calculations. Any nulls in Price or Area_sqft would be loaded as-is and silently affect revenue calculations. The Owner_Name column would remain as a combined full name with no option to split it into First_Name and Last_Name.

If **Transform Data** is chosen, the analyst can manually set Contact as a Text type to prevent incorrect numeric handling, handle any null values in Price or Area_sqft before they reach the model, split Owner_Name into First_Name and Last_Name during the transformation step, verify that all 100 rows are present and correctly structured, and only then click Close and Apply to load the fully prepared data into the model.

Question 3: What is a Fact Table and a Dimension Table? Give examples from the dataset.

Answer:

Using the **Coronavirus 2020 Dataset** which contains 500 rows and 10 columns — Country, State, Date, Confirmed_Cases, Recovered_Cases, Deaths, Active_Cases, Tests_Conducted, Hospitalized, and Vaccination_Status.

Fact Table

A Fact Table stores measurable, quantitative, and transactional data. It records what happened, capturing events or observations that can be aggregated using SUM, COUNT, or AVERAGE. Fact tables are large in size as they grow continuously with every new record.

Characteristics:

- Contains numeric measurable values called facts or measures
- Grows continuously over time
- Contains foreign keys linking to dimension tables

- Answers questions like how many, how much, and how often

Dimension Table

A Dimension Table stores descriptive and categorical information that provides context to the facts. It explains who, what, where, and when behind the numbers. Dimension tables are smaller and relatively stable in size.

Characteristics:

- Contains descriptive and categorical attributes
- Smaller and stable in size
- Has a primary key that links to the fact table
- Answers questions like which country, which state, and what date

Application to the Coronavirus Dataset

The dataset should be split into one Fact Table and supporting Dimension Tables as follows.

Fact Table — COVID_Daily_Stats

This table stores all the measurable numeric values recorded daily for each country and state combination.

Country	State	Date	Confirmed_Cases	Recovered_Cases	Deaths	Active_Cases	Tests_Conducted	Hospitalized
Brazil	State_A	43831	41240	56133	158	41424	123788	10288
UK	State_B	43832	83807	61268	1977	44002	98239	17709
Italy	State_B	43833	33434	38243	956	53744	27928	17543

Why these columns belong in the Fact Table: Confirmed_Cases, Recovered_Cases, Deaths, Active_Cases, Tests_Conducted, and Hospitalized are all numeric measurable values that represent the daily COVID-19 situation. These are the values analysts want to sum, average, and compare across countries, states, and time periods. Country, State, and Date act as foreign keys in the fact table connecting to their respective dimension tables.

Dimension Table 1 — Country_Dimension

This table stores descriptive information about each country present in the dataset.

Country_ID	Country
1	Brazil
2	UK
3	Italy
4	India
5	USA
6	Spain

Why: Country is a categorical label used to filter and group COVID statistics by nation. It is descriptive in nature and does not get summed or averaged. It gives geographical context to the facts.

Dimension Table 2 — State_Dimension

This table stores the state-level geographic information present in the dataset.

State_ID	State
1	State_A
2	State_B
3	State_C
4	State_D

Why: State is a regional categorical attribute used for sub-national level filtering and grouping of case statistics. Like Country, it is a descriptive dimension that provides geographic context.

Dimension Table 3 — Date_Dimension

This table stores time-based information derived from the Date column, enabling time-series analysis in Power BI.

Date	Month	Quarter	Year
43831	January	Q1	2020
43832	January	Q1	2020
43833	January	Q1	2020

Why: Date is the time dimension that enables trend analysis, month-over-month comparisons, and daily progression tracking of the pandemic. A dedicated Date dimension allows Power BI to support time intelligence DAX functions like rolling averages and cumulative totals.

Dimension Table 4 — Vaccination_Dimension

This table stores the vaccination status attribute which is a categorical descriptor of each record.

Vaccination_ID	Vaccination_Status
1	Yes
2	No

Why: Vaccination_Status is a categorical yes or no field describing the vaccination condition at the time of recording. It is used as a filter or slicer in reports to compare statistics between vaccinated and non-vaccinated records rather than being aggregated numerically.

Question 4: Why is Star Schema preferred over Snowflake Schema in Power BI?

Answer:

To understand why Star Schema is preferred, it is important to first understand what both schemas are and how they differ structurally.

What is Star Schema

Star Schema is a data modelling structure where a central Fact Table is directly connected to multiple Dimension Tables through one-to-many relationships. Each dimension table is denormalized, meaning all descriptive attributes are stored in a single flat table without further breakdown into sub-tables. The overall shape of the model resembles a star with the fact table at the centre and dimension tables radiating outward.

What is Snowflake Schema

Snowflake Schema is an extension of Star Schema where dimension tables are further normalized, meaning they are broken down into multiple related sub-tables to eliminate redundancy. Instead of one flat dimension table, the descriptive data is split across several layers of tables connected through additional relationships. The overall shape resembles a snowflake because of the branching structure.

Why Star Schema is Preferred in Power BI

Reason 1: Faster Query Performance Power BI uses an in-memory analytics engine called VertiPaq to process data and generate visuals. VertiPaq is specifically optimized for Star Schema structures where a visual query needs to travel only one relationship hop from a

dimension table to the fact table to retrieve results. In a Snowflake Schema, the same query must traverse multiple relationship hops across several normalized sub-tables before reaching the fact data. Each additional hop adds processing time, making Snowflake Schema noticeably slower in Power BI especially when working with large datasets like the Coronavirus dataset containing hundreds of thousands of records.

In the context of the dataset: Calculating total Deaths by Country in Star Schema requires one direct join between Country_Dimension and COVID_Daily_Stats. In Snowflake Schema the same calculation might require joining Country to Region and then Region to the Fact Table, making the query slower.

Reason 2: Simpler DAX Calculations DAX is the formula language used in Power BI to write measures and calculated columns. DAX functions such as CALCULATE, FILTER, RELATED, and USERELATIONSHIP are designed and optimized to work efficiently across single-level relationships between dimension and fact tables. When relationships span multiple normalized levels as in Snowflake Schema, DAX expressions become significantly more complex, harder to write, and more difficult to debug. Star Schema keeps all dimensional attributes in one flat table, making DAX measures straightforward and readable.

In the context of the dataset: A DAX measure calculating total Confirmed_Cases filtered by Vaccination_Status is a simple one-line measure in Star Schema. In Snowflake Schema if Vaccination_Status were stored in a sub-table linked through an intermediate table, the same measure would require additional RELATED or TREATAS functions to bridge the extra relationship levels.

Reason 3: Easier Model Management in Model View Power BI's Model View is the interface where relationships between tables are defined and visualized. Star Schema produces a clean, readable diagram with a small number of tables and a clear central structure that is easy to understand and maintain. Snowflake Schema introduces many additional sub-tables and relationship lines that make the model diagram crowded, confusing, and difficult to manage over time. For teams working collaboratively on Power BI reports, a clean Star Schema model reduces onboarding time and the risk of relationship configuration errors.

Reason 4: Reduced Relationship Complexity and Ambiguity Power BI has strict rules about relationship management including limitations on bidirectional filtering and many-to-many relationships. Every additional table and relationship introduced by Snowflake normalization increases the risk of creating ambiguous filter paths, circular dependencies, or inactive relationships that require special DAX handling. Star Schema minimizes the number of relationships in the model, reducing the chances of filter context errors that are among the most common and difficult bugs in Power BI development.

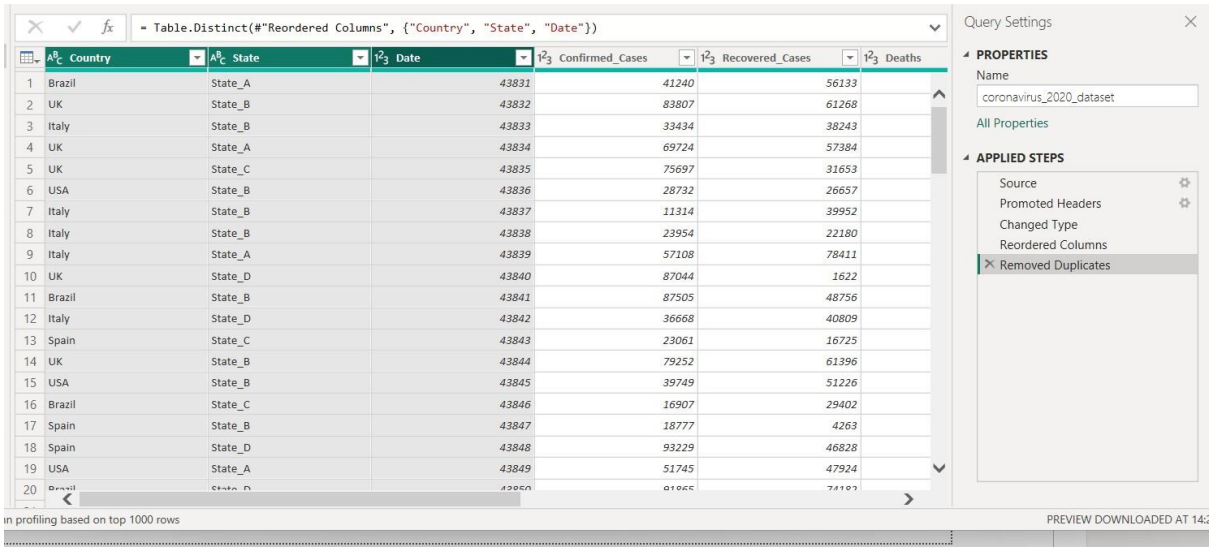
Reason 5: Better Performance with VertiPaq Compression Power BI's VertiPaq engine compresses data column by column using dictionary encoding. Denormalized dimension tables in Star Schema, despite appearing to store repeated values such as Country names, are actually compressed very efficiently by VertiPaq because repeated string values are stored as integer references in the compressed dictionary. This means the perceived storage disadvantage of denormalization in Star Schema is largely eliminated by VertiPaq compression, removing the primary technical justification for using Snowflake Schema in Power BI.

Reason 6: Faster Report Rendering for End Users When a business user interacts with a Power BI dashboard, such as clicking a slicer to filter COVID stats by Country or selecting a date range to view case trends, Power BI recalculates and re-renders all affected visuals in real time. Star Schema's simpler join paths allow these recalculations to complete faster, producing a more responsive and fluid user experience. Snowflake Schema's additional join levels slow down this recalculation, making dashboards feel sluggish especially when multiple slicers are applied simultaneously.

Practical Questions

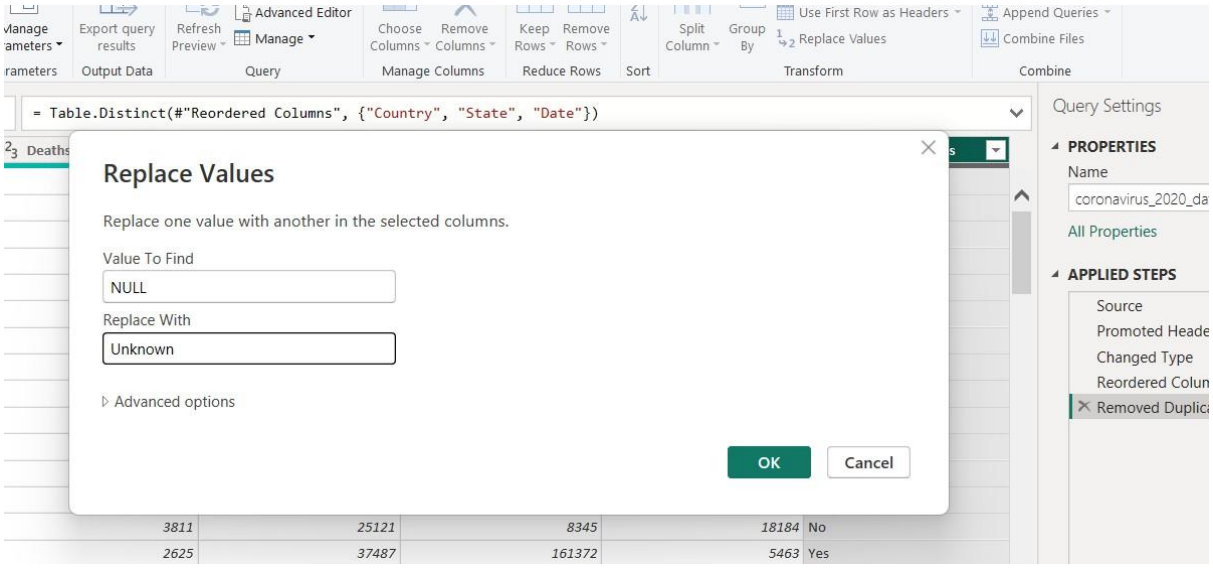
Question 5: Identify and remove duplicate records based on Date, Country, and State.

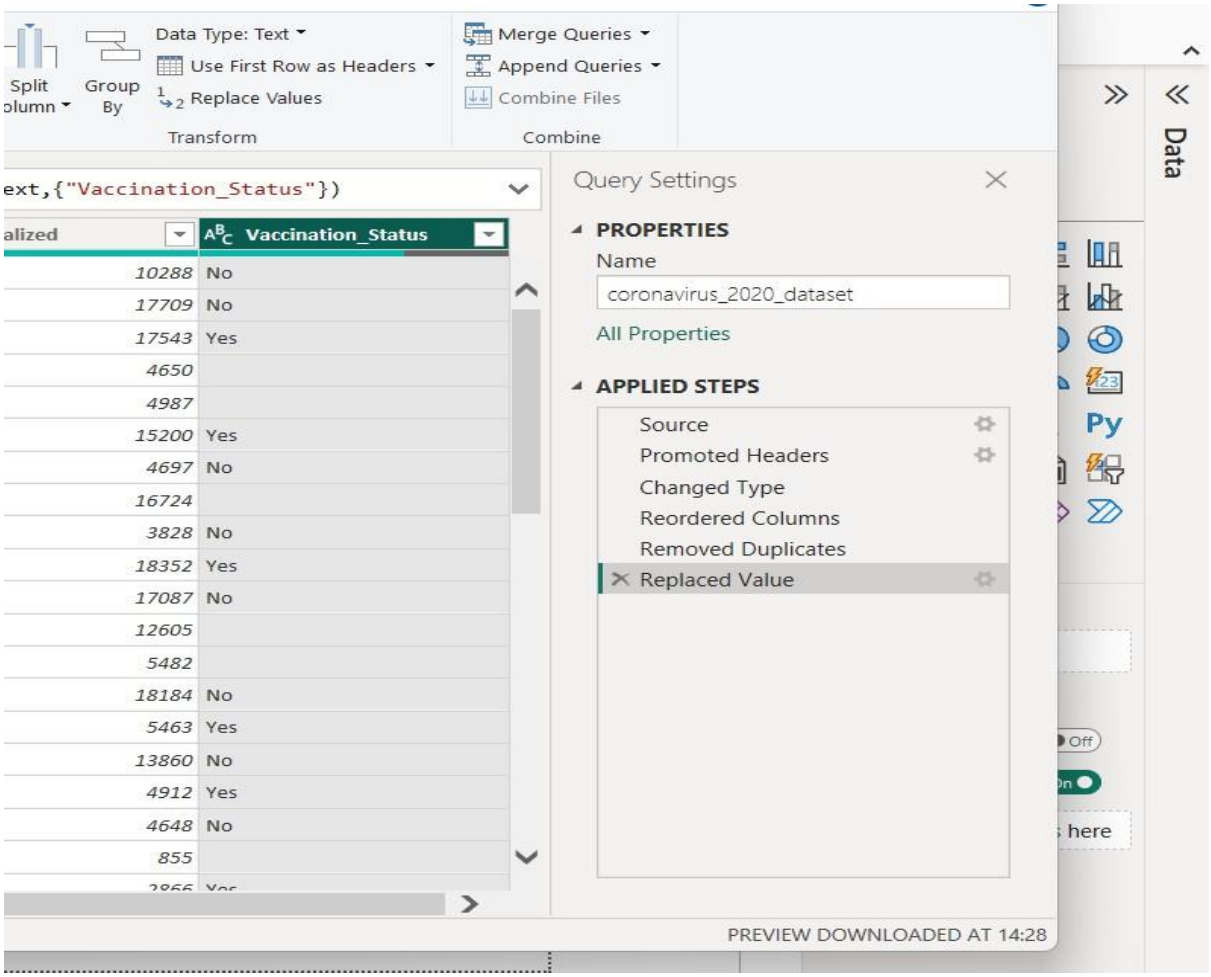
Answer:



Question 6 : Identify and replace null values in Vaccination_Status.

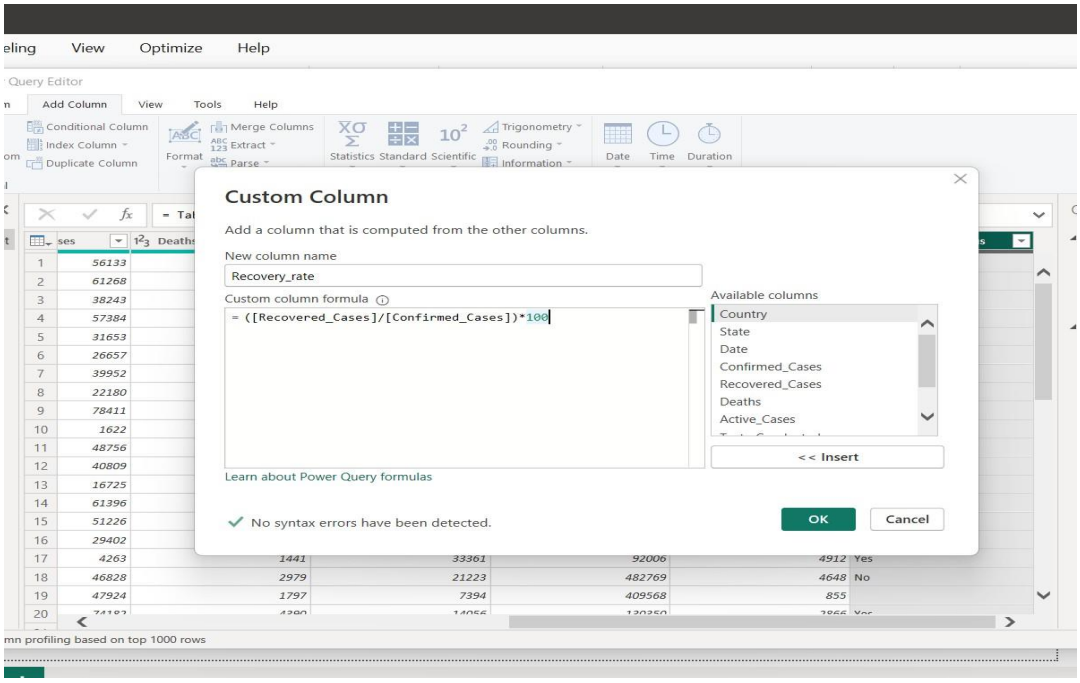
Answer:





Question 7 : Create a new column to calculate Recovery Rate.

Answer:



Question 8 : Create a summarized table showing total confirmed cases by Country.

Answer: